

Phase 3 — Solver Implementation

Scope

This document covers Requirement 7: the implementation of the abstract Integer Linear Program from Phase 1 as a runnable Python script using **PuLP** with the **CBC** solver backend, operating on the instance produced by Phase 2. It also documents one practical insight that surfaced during implementation — the structural looseness of the fatigue linearization — and the design decision that resolved it.

Overview

The solver script is a linear pipeline of pure functions, structurally identical to the preprocessing script but operating on the optimization side. Given the JSON file produced in Phase 2, it builds the ILP, invokes CBC, extracts the assignment, defensively re-verifies hard constraints, computes objective-decomposition metrics, and exports the optimized schedule as a CSV that mirrors the original dataset's schema.

The deliverables are `solver.py`, `solution.csv` (the optimized schedule), and `solver_log.json` (diagnostics: solver status, objective decomposition, solve time, and per-invigilator deviation values).

3.1 Script Architecture

The script consists of eight functions invoked sequentially by `main()`:

Function	Responsibility
<code>load_instance(path)</code>	Read <code>iap_data.json</code> and return as a dict
<code>build_model(inst, lambdas)</code>	Declare variables, add constraints, set objective — returns model + variable handles
<code>solve_model(model)</code>	Invoke CBC, return status string and solve time
<code>extract_solution(x, ...)</code>	Read integer values out of x_{ij} variables → <code>{shift: [invigilators]}</code>
<code>verify_hard_constraints(...)</code>	Defensive re-check that (H1) and (H2) hold on extracted solution

Function	Responsibility
<code>compute_metrics(...)</code>	Decompose Z^* into Z_{travel} , Z_{fatigue} , Z_{fairness} plus per-staff stats
<code>export_solution(...)</code>	Write CSV in the original dataset's schema
<code>main()</code>	Orchestrate, print progress, dump log

All paths are resolved relative to the script's own location via `Path(__file__).parent`, so the project runs from any directory without configuration.

3.2 Variable Encoding

The three families of variables defined abstractly in Phase 1 §2 translate directly to PuLP:

```
# Primary:  $x[i,j] = 1$  iff invigilator  $j$  assigned to shift  $i$ 
x = {(i, j): pulp.LpVariable(f"x_{i}_{j}", cat="Binary")}
    for i in shifts for j in invigilators}

# Fatigue auxiliary:  $y[(i,i'),j] = 1$  iff  $j$  does both back-to-back shifts
y = {(i1, i2, j): pulp.LpVariable(f"y_{i1}_{i2}_{j}", cat="Binary")}
    for (i1, i2) in E_b for j in invigilators}

# Workload deviation:  $\alpha^-, \alpha^+$  continuous  $\geq 0$ 
am = {j: pulp.LpVariable(f"am_{j}", lowBound=0) for j in invigilators}
ap = {j: pulp.LpVariable(f"ap_{j}", lowBound=0) for j in invigilators}
```

`cat="Binary"` enforces the $\{0,1\}$ domain; `lowBound=0` with no `cat` keyword gives continuous $\mathbb{R}_{\geq 0}$.

The dictionary keys are tuples — this makes constraint authoring natural ($x[i,j]$ reads exactly like x_{ij}) and avoids the overhead of a 2D list when the cardinality bound is sparse.

3.3 Hard Constraints

(H1) Coverage

```
for i, s in shifts.items():
    model += (
        pulp.lpSum(x[i, j] for j in invigilators) == s["P"],
```

```
        f"H1_cover_{i}",
    )
```

`pulp.lpSum` is the linear-summation primitive — efficient when summing many terms. The second tuple element is the constraint name (used in solver-side error reporting; PuLP requires names to be unique).

(H2) Daily Cap

```
for d, shifts_on_d in E_d.items():
    for j in invigilators:
        model += (
            pulp.lpSum(x[i, j] for i in shifts_on_d) <= daily_cap,
            f"H2_day_{d}_{j}",
        )
```

The outer loop iterates the precomputed day-to-shifts mapping E_d , so each constraint touches only shifts that actually occur on day d — no wasted zero terms.

3.4 Soft Constraint Links

(S2) Fatigue Linearization

```
for (i1, i2) in E_b:
    for j in invigilators:
        model += (
            y[i1, i2, j] >= x[i1, j] + x[i2, j] - 1,
            f"S2_b2b_{i1}_{i2}_{j}",
        )
```

Exactly the inequality $y_{i'j} \geq x_{ij} + x_{i'j} - 1$ from Phase 1 §(5.2). The structural looseness of this constraint in LP relaxation is discussed in §3.6 below.

(S3) Workload Goal

```
for j in invigilators:
    model += (
        pulp.lpSum(shifts[i]["mu"] * x[i, j] for i in shifts)
        + am[j] - ap[j] == W_bar,
```

```
        f"S3_goal_{j}",
    )
```

The shift-by-shift multiplier μ_i is pulled from the preprocessed shift dict — no per-call recomputation. The equality holds with $am[j]$ and $ap[j]$ absorbing any discrepancy.

3.5 Objective Assembly

The three penalty terms are built as separate `pulp.lpSum` expressions before being combined with the weights:

```
Z_travel = pulp.lpSum(
    d_ij.get(i, {}).get(j, 0) * x[i, j]
    for i in shifts for j in invigilators
)
Z_fatigue = pulp.lpSum(y[i1, i2, j] for (i1, i2) in E_b for j in invigilators)
Z_fairness = pulp.lpSum(am[j] + ap[j] for j in invigilators)

model += (
    lambdas["travel"] * Z_travel
    + lambdas["fatigue"] * Z_fatigue
    + lambdas["fairness"] * Z_fairness
)
```

`d_ij.get(i, {}).get(j, 0)` reads the sparse travel-cost dict with a default of 0 for entries not stored (home-match case). The objective expression is added to the model directly — PuLP infers it's the objective from the absence of a relational operator.

3.6 Solver Configuration

The model is invoked as:

```
solver = pulp.PULP_CBC_CMD(msg=False, gapRel=0.05, timeLimit=60)
status = model.solve(solver)
```

The choice of `gapRel=0.05` (a 5% relative optimality gap) is the central design decision of this phase, and warrants explanation.

 **Why this gap is necessary**

Initial runs with default settings (provable-optimality termination) timed out after 30+ seconds without proving optimality. Diagnostic runs with verbose CBC output revealed the cause: the LP relaxation of the fatigue linearization is structurally loose.

Specifically, the constraint $y_{ii'j} \geq x_{ij} + x_{i'j} - 1$ allows $y = 0$ whenever both x -values are in the LP-fractional range. If LP relaxation sets $x_{ij} = x_{i'j} = 0.5$, then $y_{ii'j} \geq 0$ is trivially satisfied, and the LP "lies" about the cost of that potential fatigue event. The LP lower bound therefore systematically underestimates the integer optimum, forcing branch-and-bound to compensate by exploring many nodes.

Empirically, CBC found the optimal incumbent within ≈ 7 seconds, then spent another ≈ 53 seconds shaving the LP-integer gap from 4% down to 0%. The schedule did not change during that interval — only the proof of optimality.

The practical fix is to **accept** the looseness rather than fight it: instruct CBC to terminate as soon as the incumbent is provably within 5% of the LP lower bound. This:

- Produces the same schedule that exhaustive search would (in our experiments, the incumbent found at ~ 7 seconds matches the 60-second result).
- Reduces solve time from ~ 60 seconds to ~ 5 – 10 seconds, an order of magnitude.
- Preserves the rigor of the deliverable: Z^* remains within a quantified bound of the true optimum.

A tighter formulation that pre-empts the LP looseness — for instance, adding $y_{ii'j} \leq x_{ij}$ and $y_{ii'j} \leq x_{i'j}$ as upper-bound constraints — does not help, because those constraints constrain y from above, while LP needs to be forced *upward* (which only integer-feasibility can do). The looseness is intrinsic to the AND-linearization pattern, not a bug in our formulation.

3.7 Solution Extraction and Verification

After CBC returns, the integer assignment is extracted from the x variables and converted to a `{shift_id: [invigilator_ids]}` dict:

```
def extract_solution(x, shifts, invigilators):
    assignment = defaultdict(list)
    for i in shifts:
        for j in invigilators:
            if pulp.value(x[i, j]) > 0.5:
```

```
assignment[i].append(j)
return dict(assignment)
```

The `> 0.5` test is a floating-point tolerance: CBC returns integer values to within $\sim 10^{-9}$, so any threshold strictly between 0 and 1 works.

A separate `verify_hard_constraints` function then re-checks (H1) and (H2) on the extracted assignment from first principles. This defensive step is cheap (linear in the assignment size) and catches three classes of errors: (a) silent solver failures, (b) bugs in the model where constraints don't actually enforce what they claim, and (c) extraction bugs in the threshold logic.

If this check passes, the schedule is by construction feasible — independent of whether the solver was correct.

3.8 Output Format

`solution.csv`

The CSV mirrors the original dataset's nine columns in Vietnamese: `Ca thi`, `Ngày`, `GIỜ`, `MS`, `Ca thi`, `Nhiệm vụ`, `MS của CÁN BỘ COI THI`, `Thời gian`, `Thứ`, `Cơ sở`. This makes side-by-side comparison in Phase 4 a one-line `pd.merge` operation.

Treatment of `Nhiệm vụ` in the output

Because the model treats all three roles (CBCT, Thư ký, Trưởng HĐ) as equivalent invigilation duty (a Phase 1 design decision), the output normalizes every assignment's `Nhiệm vụ` to the campus-prefixed CBCT code (`LTK_CBCT` or `DiAn_CBCT`). This faithfully represents what the model actually optimized — undifferentiated invigilation — without inventing role attributions the model never decided.

`solver_log.json`

Captures everything needed for downstream analysis without re-running the solver:

```
{
  "status":          "Optimal",
  "objective":        Z* value,
  "z_travel":         travel-term contribution,
  "z_fatigue":         fatigue-term contribution,
  "z_fairness":        fairness-term contribution,
  "lambdas":          {travel, fatigue, fairness},
```

```

"solve_time_sec":    wall time,
"n_variables":       total decision variables,
"n_constraints":     total constraints,
"count_per_staff":   {cb_id: number of shifts},
"weighted_workload_per_staff": {cb_id: sum of mu_i},
"alpha_minus":       {cb_id: under-deviation},
"alpha_plus":        {cb_id: over-deviation}
}

```

This is the primary input to Phase 4's comparison work.

Realized Results on the HCMUT Instance

With the initial weight configuration $(\lambda_1, \lambda_2, \lambda_3) = (1, 5, 1)$:

Quantity	Value
Solver status	Optimal (within 5% gap)
Total objective Z^*	≈ 194.88
Z_{travel}	181.50
Z_{fatigue}	0
$Z_{\text{fairness}} (\sum_j \alpha_j^- + \alpha_j^+)$	≈ 13.38
Variables	6,643
Constraints	3,569
Solve time	≈ 5.4 s
Hard-constraint verification	✓ all satisfied

A high-level comparison against the baseline is shown below; the deeper analysis (including weight tuning, per-staff distributions, and Pareto-frontier exploration) is the subject of Phase 4.

Metric	Baseline	Solver	Change
Workload count range	3 to 19	10 to 11	std dev 2.22 $\rightarrow$ 0.50
Back-to-back fatigue events	129	0	−100%
Travel cost (synthetic homes)	385.0	181.5	−52.9%

The model improves substantially on all three optimization dimensions while preserving total demand (the same 769 invigilator-shift assignments, redistributed). The fatigue elimination is

exact — the assignment contains zero same-day consecutive double-shifts. This is not a coincidence of the dataset; it is enforced by the fatigue term having the largest weight in the objective, combined with sufficient slack in the daily cap (which permits 4 shifts/day but does not require any of them to be consecutive).

Summary

Stage	Implementation file	Output
Phase 1 abstract model	(specification only)	—
Phase 2 instance	<code>preprocess.py</code>	<code>iap_data.json</code>
Phase 3 solver	<code>solver.py</code>	<code>solution.csv</code> , <code>solver_log.json</code>
Phase 4 analysis	(next)	comparison report, tuning plots

Phase 3 leaves the model in a state where the weight triple $(\lambda_1, \lambda_2, \lambda_3)$ is a tunable knob. Phase 4 will sweep this triple, characterize the trade-off frontier, and defend a chosen operating point on practical grounds.